CS9711 Fingerprint Installer for Debian 13 — English guide

Repository: github.com/StrangerKLG/cs9711-debian-installer

Main guide

CS9711 Fingerprint Installer for Debian 13

Experimental installer for USB fingerprint scanners detected as:

```
2541:0236 Chipsailing CS9711Fingprint
```

Tested with this specific USB fingerprint reader bought by the maintainer:

- <https://www.ozon.ru/product/usb-skaner-otpechatkov-paltsev-windows-hello-dl-ya-pk-noutbuka-chernoe-1410552111/>

This is **not advertising or an affiliate recommendation**. It is only a reference to the exact device that was purchased and tested.

It automates the working Debian 13 flow tested on a live system:

- install build/runtime dependencies;
- build pinned `ericlinagora/libfprint-CS9711` commit `c242a40fcc51aec5b57d877bdf3edfe8cb4883fd`;
- patch old Meson `udev` dependency to Debian 13 `libudev`;
- install `libfprint` into `/usr/local`;
- add udev rule to disable USB autosuspend for `2541:0236`;
- optionally enable fingerprint authentication for `sudo`, SDDM, and KDE/Polkit admin prompts, with password fallback.

PDF guides

Direct PDF download links:

- ENG PDF guide
- RU PDF инструкция

If GitHub opens a preview page instead of downloading the PDF, download it from the terminal:

```
wget -O RU_cs9711-debian-installer-guide.pdf
https://raw.githubusercontent.com/StrangerKLG/cs9711-debian-
installer/master/docs/pdf/RU_cs9711-debian-installer-guide.pdf
```

A real PDF file starts with `%PDF`. If the downloaded file starts with `<!DOCTYPE html>`, the browser saved a GitHub web page instead of the PDF.

Russian documentation

Russian documentation is available in `docs/ru/README.ru.md`.

AI-generated installer disclosure

This installer script and documentation were drafted by an AI agent, with human direction and review, based on manual testing on Debian 13.

The AI agent did **not** create the fingerprint driver. This repository wraps and automates installation of the community fork `ericlinagora/libfprint-CS9711`, pinned to the tested commit `c242a40fcc51aec5b57d877bdf3edfe8cb4883fd`. See Credits and related projects and Third-party notices.

Please read the script before running it. It modifies `/usr/local` libraries, udev rules, and optionally PAM files.

Big warning

This is not an official Debian/libfprint package. It installs a forked `libfprint` into `/usr/local` so `fprintd` loads it before the stock library.

This repository does not vendor or redistribute the upstream driver source or binaries. The installer downloads and builds upstream `libfprint-CS9711` locally. The wrapper files in this repository are MIT-licensed, but upstream `libfprint`/`libfprint-CS9711` remains under its own LGPL-2.1 license.

Do **not** enable fingerprint globally in `/etc/pam.d/common-auth` unless you know exactly what you are doing. In the tested KDE setup, KDE lockscreen fingerprint crashed `fprintd`. The installer deliberately uses targeted PAM snippets for `sudo`, SDDM, and optional Polkit instead.

Keep a password/root/SSH recovery path open while testing.

Step-by-step for absolute beginners

This section is intentionally very explicit.

1. Open the terminal

After logging into KDE, open the application menu and start **Konsole** / **Terminal**.

You should see a window with a prompt similar to:

```
stranger@cs9711-github:~$
```

2. For VirtualBox tests only: attach the USB fingerprint reader

If you are testing inside VirtualBox, the physical USB scanner must be passed through to the VM.

In the VirtualBox VM window menu:

```
Devices → USB → Chipsailing CS9711Fingprint
```

or choose the USB device with ID:

```
2541:0236
```

Make sure the checkbox near this USB device is enabled. After that the reader is connected to the virtual machine, not to the host system.

Expected result inside the VM:

```
lsusb
```

should show a line containing:

```
2541:0236
```

If you install on a real physical Linux machine, do **not** do this step. There is no VirtualBox USB passthrough on a real machine; the scanner only needs to be plugged into USB.

Optional for VirtualBox: install Guest Additions for copy-paste

If copy-paste between the host and the VM does not work, install VirtualBox Guest Additions inside the VM.

Skip this step on a real physical Linux machine.

In the VirtualBox VM window menu:

```
Devices → Insert Guest Additions CD image...
```

Then run in the VM terminal:

```
sudo apt update
sudo apt install -y build-essential dkms linux-headers-amd64 bzip2 perl make gcc wget
sudo mkdir -p /mnt/vboxga
sudo mount /dev/sr0 /mnt/vboxga
```

```
sudo sh /mnt/vboxga/VBoxLinuxAdditions.run
sudo reboot
```

After reboot, enable:

```
Devices → Shared Clipboard → Bidirectional
```

3. Copy and paste the project download commands

Copy this whole block and paste it into the terminal:

```
git clone https://github.com/StrangerKLG/cs9711-debian-installer.git
cd cs9711-debian-installer
```

Press Enter if the terminal does not start automatically.

Expected result: Git downloads the repository, then the prompt changes so the current folder ends with:

```
cs9711-debian-installer$
```

4. Run the first install step

Copy and paste:

```
sudo ./install.sh --user "$USER" --driver-only --enroll --verify --yes
```

The terminal will ask for your password. Type your Linux login password and press Enter.

Important: while typing the password, nothing may be shown on screen. This is normal.

The script will install packages, build the driver, and then start fingerprint enrollment.

5. Enroll the finger

When the terminal asks to touch the fingerprint reader, place the same finger on the reader several times. Move it slightly between touches.

Look for a successful verification result:

```
verify-match
```

If you see `verify-no-match`, repeat the enrollment step.

6. Enable fingerprint login/auth prompts

Only after `verify-match`, copy and paste:

```
sudo ./install.sh --user "$USER" --no-driver --sudo --sddm --polkit --yes
```

Expected result: the script finishes with `Done` and prints recommended checks.

7. Test sudo

Copy and paste:

```
sudo -k && sudo true
```

Expected result: the system asks for the enrolled finger. Touch the reader. If fingerprint fails or times out, enter the password.

If copy-paste merged commands together

Some PDF viewers may paste line breaks incorrectly. If you see an error like:

```
sudo: truesudo: command not found
```

then two commands were pasted as one broken word, for example `sudo truesudo`.

Do not continue with the broken command. Copy and run this single-line command instead:

```
sudo -k && sudo true
```

In general, when a command block contains several lines, paste it carefully and check that every line starts where it should. If unsure, use the single-line commands shown in this guide.

8. Test SDDM login

Log out of KDE. On the login screen, select the user, press Enter in the password field, then touch the enrolled finger.

Expected result: you log in without typing the password. The password must still work as fallback.

9. Test KDE/Polkit admin prompt

Open a KDE settings action that asks for administrator authentication. When the dialog says “Authentication is required”, touch the enrolled finger.

Expected result: the admin prompt accepts the fingerprint. The password must still work as fallback.

Quick start

Clone or download this project, then:

```
sudo ./install.sh --user "$USER" --driver-only --enroll --verify
```

If you get `verify-match`, optionally enable fingerprint for `sudo`, SDDM login, and KDE/Polkit admin prompts:

```
sudo ./install.sh --user "$USER" --no-driver --sudo --sddm --polkit
```

Test `sudo`:

```
sudo -k && sudo true
```

For SDDM: log out to the login screen, press Enter on the password field, then touch the enrolled finger. Password should remain a fallback.

For Polkit/KDE admin prompts: trigger a desktop action that opens “Authentication is required” / “Требуется аутентификация”, then touch the enrolled finger. Password should remain a fallback.

One-shot install

For confident testers only:

```
sudo ./install.sh --user "$USER" --sudo --sddm --polkit --enroll --verify
```

This still avoids global `common-auth` fingerprint auth.

Options

```
--user USER       Target login user
--finger FINGER   Finger name, default right-index-finger
--driver-only     Driver + udev only; no PAM changes
--sudo            Enable sudo fingerprint auth for USER
--sddm            Enable SDDM fingerprint auth for USER
--polkit          Enable KDE/Polkit admin-prompt fingerprint auth for USER
--enroll          Run fprintd-enroll
--verify          Run fprintd-verify
--no-driver       Skip driver build/install
--no-udev         Skip udev autosuspend rule
-y, --yes         Non-interactive prompts
--rollback        Print rollback notes
```

What gets changed

- `/usr/local/lib/.../libfprint-2.so*`
- `/etc/ld.so.conf.d/local-libfprint.conf`
- `/etc/udev/rules.d/99-cs9711-fingerprint-power.rules`

- optionally `/etc/pam.d/sudo`
- optionally `/etc/pam.d/sddm`
- optionally `/etc/pam.d/polkit-1` (local override copied from `/usr/lib/pam.d/polkit-1` when needed)

Backups are stored under:

```
/root/cs9711-installer-backups/YYYYMMDD-HHMMSS/
```

Rollback

Show rollback notes:

```
sudo ./install.sh --rollback
```

Short version:

1. Restore `/etc/pam.d` from the backup folder, or remove blocks between:

```
# BEGIN cs9711-fingerprint-installer
# END cs9711-fingerprint-installer
```

2. Disable local libfprint override:

```
sudo rm -f /etc/ld.so.conf.d/local-libfprint.conf
sudo mkdir -p /root/libfprint-cs9711-disabled
sudo find /usr/local/lib /usr/local/lib64 -name 'libfprint-2.so*' -exec mv -t
/root/libfprint-cs9711-disabled/ {} + 2>/dev/null || true
sudo ldconfig
sudo systemctl restart fprintd || true
```

3. Remove udev rule if desired:

```
sudo rm -f /etc/udev/rules.d/99-cs9711-fingerprint-power.rules
sudo udevadm control --reload-rules
sudo udevadm trigger -s usb || true
```

Tested environment

- Debian 13 trixie
- `fprintd 1.94.5-2`
- stock `libfprint-2-2 1:1.94.9-1` did **not** support the scanner
- scanner USB ID `2541:0236`
- fork commit `c242a40fcc51aec5b57d877bdf3edfe8cb4883fd`
- `sudo` fingerprint auth: works

- SDDM login fingerprint auth: works via Enter → finger
- KDE/Polkit admin-prompt fingerprint auth: works
- KDE lockscreen fingerprint: not recommended; crashed `fprintd` during testing

Why not a `.deb` yet?

A proper package would be better, but the first goal is to reduce a 20+ page manual guide to a safe, auditable script. A `.deb`/CI-built release can be added later.

Clean VM test guide

See docs/quick-test-vm.md.

Copy-paste clean VM instructions

For a fresh Debian 13 VM test, see docs/copy-paste-clean-vm.md.

Third-party notices

Third-party notices

This repository contains a small installer script and documentation. It does **not** vendor, copy, or redistribute the `libfprint`, `fprintd`, or `libfprint-CS9711` source code.

The installer downloads and builds the following upstream project on the user's machine:

- `ericlinagora/libfprint-CS9711`

- URL: <https://github.com/ericlinagora/libfprint-CS9711>

- Tested pinned commit: `c242a40fcc51aec5b57d877bdf3edfe8cb4883fd`

- Upstream license: GNU Lesser General Public License v2.1 (`LGPL-2.1`), inherited from `libfprint`

Related upstream projects:

- `libfprint`: <https://gitlab.freedesktop.org/libfprint/libfprint>
- `fprintd`: <https://gitlab.freedesktop.org/libfprint/fprintd>

License separation

The files in this repository are licensed under the repository `LICENSE` file.

That license applies only to this installer/documentation wrapper. It does **not** relicense upstream `libfprint`, `fprintd`, or the `libfprint-CS9711` fork. Those projects remain under their own licenses and copyright notices.

When the installer builds and installs `libfprint-CS9711`, the resulting library is governed by the upstream LGPL-2.1 terms. Users should review the upstream license before using or redistributing the built library.

No binary redistribution

This repository does not publish prebuilt binaries of `libfprint-CS9711` or `libfprint`. It only automates a local build from the pinned upstream source commit.

Credits

Credits and related projects

This project is a small installer/documentation wrapper around existing open-source fingerprint work.

Core upstream projects

- `libfprint` upstream: <https://gitlab.freedesktop.org/libfprint/libfprint>
- `fprintd` upstream: <https://gitlab.freedesktop.org/libfprint/fprintd>
- Community CS9711 fork used by this installer: <https://github.com/ericlinagora/libfprint-CS9711>

- Pinned commit tested here: `c242a40fcc51aec5b57d877bdf3edfe8cb4883fd`

Tested hardware reference

The tested scanner was purchased here:

- <https://www.ozon.ru/product/usb-skaner-otpechatkov-paltsev-windows-hello-dl-ya-pk-noutbuka-chernoe-1410552111/>

This is **not advertising or an affiliate recommendation**. It is only a reference to the exact device bought and tested by the maintainer.

Documentation references

- ArchWiki fprint: <https://wiki.archlinux.org/title/Fprint>
- Debian PAM documentation: <https://wiki.debian.org/PAM>

AI disclosure

The installer script and documentation in this repository were drafted by an AI agent, with human direction and review, based on manual testing logs and a verified Debian 13 setup.

The AI agent did not create the fingerprint driver. The CS9711 driver support comes from the community fork listed above. This project only automates and documents a tested installation flow.

Before running the script, read it. It modifies system libraries, udev rules, and optionally PAM configuration.

Clean VM test guide

Clean Debian VM test guide

This guide is for testing the installer in a clean Debian 13 VM.

1. Install Debian 13 in VirtualBox

Recommended VM settings:

- OS: Debian 64-bit
- RAM: 2-4 GB
- CPU: 2 cores
- Disk: 25+ GB
- USB Controller: USB 3.0/xHCI
- Add USB filter for the fingerprint scanner:

```
Vendor ID: 2541
Product ID: 0236
Name: Chipsailing CS9711Fingprint
```

For a simple text-console VM, if VirtualBox graphics is broken on KDE/Wayland, use:

Guest GRUB after install:

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet nomodeset"
GRUB_TERMINAL=console
GRUB_GFXPAYLOAD_LINUX=text
```

Then run:

```
sudo update-grub
sudo reboot
```

1.1 Attach the USB scanner to the running VM

For VirtualBox testing, plugging the scanner into the host is not enough. Attach it to the VM:

```
VirtualBox VM window → Devices → USB → Chipsailing CS9711Fingprint
```

or select the device with USB ID `2541:0236`.

The menu item should become checked. Then verify inside the VM:

```
lsusb
```

Expected: one line contains `2541:0236`.

On a real physical Linux machine, skip this whole VirtualBox USB passthrough step.

1.2 Optional: install VirtualBox Guest Additions for clipboard

This step is useful only for VirtualBox testing. It enables shared clipboard, better mouse integration, and display integration.

If you install on a real physical Linux machine, skip this step.

In the VirtualBox VM window menu, choose:

```
Devices → Insert Guest Additions CD image...
```

Then inside the VM open Konsole and run:

```
sudo apt update
sudo apt install -y build-essential dkms linux-headers-amd64 bzip2 perl make gcc wget
sudo mkdir -p /mnt/vboxga
sudo mount /dev/sr0 /mnt/vboxga
sudo sh /mnt/vboxga/VBoxLinuxAdditions.run
sudo reboot
```

After reboot, log in again. In VirtualBox settings/menu, make sure shared clipboard is enabled:

```
Devices → Shared Clipboard → Bidirectional
```

Expected result: copying text from the host and pasting it into the VM terminal works.

Note: Debian 13/trixie repositories may not provide `virtualbox-guest-utils` / `virtualbox-guest-x11` packages by default. The Guest Additions CD image method above is the tested path.

2. Install git and download this project

Inside the VM:

```
sudo apt update
sudo apt install -y git ca-certificates
cd ~
git clone https://github.com/OWNER/REPO.git cs9711-debian-installer
cd cs9711-debian-installer
```

Replace `OWNER/REPO` with the final GitHub repository path.

3. Driver + enroll + verify

```
sudo ./install.sh --user "$USER" --driver-only --enroll --verify --yes
```

During enrollment, touch the same finger repeatedly, lifting it between touches and slightly changing position.

Expected success markers:

```
Enroll result: enroll-completed
Verify result: verify-match (done)
```

4. Enable sudo fingerprint auth

Only after `verify-match`:

```
sudo ./install.sh --user "$USER" --no-driver --sudo --yes
```

Test it safely:

```
sudo -k && sudo true
```

Expected behavior: sudo asks for the enrolled finger. If fingerprint fails or times out, password remains fallback.

Optional KDE/Polkit admin prompts on desktop installs:

```
sudo ./install.sh --user "$USER" --no-driver --polkit --yes
```

This enables fingerprint in Polkit authorization dialogs only; it does not enable KDE lockscreen or wallet fingerprint.

5. Optional: SDDM login

Only if you use SDDM and understand the caveat about KDE lockscreen:

```
sudo ./install.sh --user "$USER" --no-driver --sddm --yes
```

Test by logging out to SDDM, pressing Enter on the password field, then touching the enrolled finger.

Do not test with apt install

Use this instead:

```
sudo -k && sudo true
```

It tests authentication without changing packages.

Troubleshooting: pasted command became `truesudo`

If the terminal says:

```
sudo: truesudo: command not found
```

then the PDF viewer or clipboard merged two lines into one. Run the sudo test as one explicit line:

```
sudo -k && sudo true
```